

Q2: building the flood model responsibly

Answering "will this be correct, or are we playing with someone's safety?"

Short answer

Your architecture is right — observed reports as the primary answer, model as a supplementary layer. That's how real flood warning systems are actually built.

But a disclaimer is **not** what makes it safe. People don't read disclaimers, and someone deciding whether to ride through an underpass at 7am reads them least of all.

What makes it safe is one design rule.

The rule that matters

The model may only ever **ADD** a warning. It may never **REMOVE** one, and it may never say "clear."

Look at the two ways the model can be wrong, and what each costs a real person:

Model is wrong	What happens	Cost
False positive — says risky, actually fine	Someone takes a different road	10 minutes
False negative — says fine, actually flooded	Someone rides a two-wheeler into opaque standing water	Potentially their life

These are not comparable, and **that asymmetry is the whole ethical question.**

So: the absence of a warning must never be presented as an assurance. There is no green. There is no "safe route." There is no "this area is clear." The map shows warnings and it shows nothing — never reassurance.

Get that right and a wrong model output can only ever cost someone an inconvenience. **You've made the failure mode harmless by design, instead of hoping the model is accurate.** That's what actually discharges the responsibility, and it holds even if the model turns out to be bad.

Three supporting rules

- 1. Observed and modelled never share a visual language.** Different colour, different symbol, different wording — and always labelled at the point of display, not in a footer.
 - Observed: "17 residents reported flooding here, most recently 14 July."
 - Modelled: "Terrain suggests water may collect here. Not verified by any report."A user must never have to work out which kind of statement they're reading.
- 2. No false precision.** No depths. No percentages. No "68% probability." Ordinal bands only — *higher risk / lower risk / unknown*. A number implies a confidence the model does not have, and precision is itself a form of dishonesty here.
- 3. Reports outrank the model, always.** If residents report flooding somewhere the model says is fine, the report wins and is displayed. The model never contradicts an observation — it only fills in where there are no observations yet.

Why this is the stronger pitch, not a hedged one

Most teams will show a flood prediction and claim accuracy. You show one and say:

"Our model can only add warnings, never remove them. A false alarm costs someone ten minutes. A missed warning could cost someone their life. Those aren't comparable, so we designed the system so that being wrong can only ever inconvenience you."

Very few student teams have thought about asymmetric error costs. A professor will notice immediately, and a municipal official — who has to think about liability every day — will notice harder.

It also settles the liability question before anyone asks it: you are advisory, you never assure, and you're explicit about which claims are observed versus modelled.

How hard is it actually to build?

With AI writing the code, realistically **4-6 hours**, and the time is not in the typing:

Step	Time	Where it hurts
DEM download + reprojection	30-60 min	CRS mismatches, nodata handling
Pit-fill → flow direction → flow accumulation	1-2 hr	The classic geospatial debugging sink
Extract depressions, rank by upstream area	1 hr	Tuning what counts as a sink
Join to OSM roads and junctions	1 hr	Geometry snapping
Ordinal ranking + output layer	30 min	Straightforward

Do this 20-minute check before committing the 4-6 hours. Download the DEM and compute relief statistics across the city — elevation range, mean slope. **If Nagpur is too flat, flow accumulation returns mush:** sinks everywhere, no coherent channels, and the output is noise dressed as analysis. I flagged this earlier and it is still unverified.

If the terrain check fails, you ship reported-locations only and say why — which is a perfectly good answer, and an honest one.

Where it belongs in the build

Second. After Q1 is complete and working.

Q1 is your project and it's nearly done. Q2 is a 4-6 hour bet on terrain you haven't verified, in a 24-hour window. Build it when Q1 is finished, time-box it, and if it isn't working by your cutoff, ship the reported-locations layer alone.

Losing the model costs you one feature. Losing Q1 because you spent the night debugging projections costs you the hackathon.

In one line

Not a disclaimer — an asymmetry. The model adds warnings and never removes them, so when it's wrong, the worst it can do is send someone the long way round.